MERN Stack Technical Drill: Batch 1

Frontend (React) & API Communication (Q1 - Q20)

REACT & CLIENT-SIDE LOGIC

1. Why is React preferred over traditional JS for MERN applications?

*"React uses a **Virtual DOM** and a component-based architecture. This allows for efficient UI updates and high reusability, which is critical when building complex interfaces like your LeadGen dashboard or a MERN-based tourist site."*

2. How do you handle 'State Persistence' in a MERN application?

*"For temporary persistence (like form data), I use **LocalStorage** or **SessionStorage**. For global app state (like user auth), I use **Redux** or the **Context API**. This ensures the user stays logged in even after a page refresh."*

3. Explain 'Client-Side Routing' vs 'Server-Side Routing'.

*"In MERN, we use **React Router** for client-side routing. The page doesn't reload from the server; instead, React swaps components dynamically. This creates a Single Page Application (SPA) feel that is much faster than traditional server-side rendering."*

4. How did you implement User Auth in your MERN Tourist Website?

*"I used **JWT (JSON Web Tokens)**. The backend generates a token upon successful login, which the React frontend stores in LocalStorage or an HttpOnly cookie. Every subsequent API request includes this token in the 'Authorization' header to verify the user."*

API INTERACTION (AXIOS & FETCH)

5. Why use Axios instead of the native Fetch API?

*"Axios provides automatic JSON transformation, built-in **Interceptors** (useful for adding tokens to every request), and better error handling. It also supports request cancellation and has a simpler syntax for handling timeouts."*

6. What is an 'API Interceptor' and how did you use it?

*"An interceptor is a function that runs before a request is sent or after a response is received. I use it to globally attach the **JWT token** to every outbound request, ensuring I don't have to manually add it in every component."*

7. How do you handle 'Race Conditions' when fetching data in React?

"I use a 'cleanup' flag in `useEffect` *or an* ***AbortController****. If the component unmounts or the search query changes before the previous request finishes, I cancel the old request to prevent updating the state with stale data."*

OPTIMIZATION & SECURITY

8. What is CORS and how do you resolve it in a MERN stack?

"CORS (Cross-Origin Resource Sharing) is a browser security feature. Since React (Port 3000) and Express (Port 5000) run on different ports, I use the `cors` *middleware in the Node.js backend to explicitly allow the React origin."*

9. Explain 'Prop Drilling' and why it's a bottleneck in MERN.

"Prop drilling is passing data through components that don't need it. In a large MERN app, this makes code hard to maintain. I solve this by using ***Redux Toolkit*** *to provide a global store accessible by any component."*

10. How do you implement 'Lazy Loading' for heavy MERN routes?

"I use `React.lazy()` *and* `Suspense`*. This splits the JavaScript bundle, so the browser only loads the code for the specific page the user is visiting, significantly improving initial load time."*

ADVANCED PATTERNS

11. What is the 'Proxy' setting in package.json?

"In development, setting `"proxy": "http://localhost:5000"` *in the React* `package.json` *allows us to call API endpoints using relative paths like* `/api/users`*, avoiding CORS issues during local development."*

12. How do you handle image uploads in a MERN application?

"I use ***Multer*** *middleware on the backend to handle* `multipart/form-data`*. For storage, I prefer cloud services like* ***Cloudinary*** *or* ***AWS S3*** *rather than storing images directly on the server to ensure scalability."*

13. Explain 'Context API' vs 'Redux'.

"Context API is built-in and perfect for small apps or static data (like themes). Redux is a robust state management library with middleware support and powerful debugging tools, ideal for complex MERN apps with high-frequency updates."

14. What are 'React Hooks'? Which ones are vital for MERN?

"Hooks are functions that let you use state and lifecycle features. ***useState*** *for local data,* ***useEffect*** *for API calls, and* ***useContext*** *for global state are the core hooks used in almost every MERN component."*

15. How do you protect 'Private Routes' in React?

*"I create a **Higher-Order Component (HOC)** or a wrapper component that checks for the presence of an auth token. If the user isn't logged in, it redirects them to the login page using the* `<Navigate />` *component."*

16. What is 'Redux Thunk' or 'Redux Saga'?

*"They are middlewares that allow you to write **Asynchronous logic** (like API calls) inside Redux. Thunk is simpler and uses functions; Saga uses Generator functions and is better for complex side-effect management."*

17. Explain 'Environment Variables' on the frontend.

"I use `.env` *files to store the **Backend Base URL**. In React, these must start with* `REACT_APP_`*. This allows the app to switch between local development and production URLs automatically."*

18. What is 'Error Boundary' in React?

"It's a component that catches JavaScript errors anywhere in its child component tree and displays a fallback UI instead of crashing the whole app. It's essential for a professional user experience."

19. How do you perform 'Debouncing' in a search bar?

"I use a timer (`setTimeout`*) inside* `useEffect`*. The API call only triggers if the user stops typing for a set duration (e.g., 500ms), which saves server resources and prevents excessive re-renders."*

20. Why use 'Functional Components' over 'Class Components'?

"Functional components are more concise, easier to test, and support Hooks. They are the modern standard for React development, reducing boilerplate code and improving performance through better optimization."